Zenith: learning what to investigate before forecasting

Will Cook • 12 October 2026

Abstract. Zenith treats reasoning as a policy for choosing useful numerical investigations. An initial forecast may miss a distinction between sources, feature families or times. A language-model programme proposes a small set of questions, numerical tools answer them using earlier data, and a final decision may revise the forecast. Later errors guide one reusable instruction change, evaluated on separate dates. We implement this loop for six research baskets sharing the histories of 147 financial instruments and 17 market indices. Comparisons include an all-universe numerical model, fixed investigations and a single controller with the same data and tools. The study is a small retrospective execution test; its question is whether selective investigation earns its additional computation.

1. The task. At each cutoff, predict the five-market-session return of every basket using the entire declared universe. A basket's own members are only one possible source of information: breadth in another group, lagged returns or changing volume may be worth testing. Such patterns are empirical hypotheses. Neither correlated prices nor a dependency between computations identifies an economic cause.

The source is the adjusted-instrument and index history published by SocrateAI under a dataset-specific MIT-form licence [1]. We pin its repository commit and import through 31 December 2025. The audited panel contains 447,243 instrument observations and 59,319 index observations; 2,319 rows with missing index prices are quarantined. Optional blank volume values remain missing.

Membership is fixed on 29 December 2021, using only preceding observations: at least 95% quote coverage over 252 sessions, a quote at formation and positive median volume. We sort the 147 eligible instruments by their preceding 126-session beta to the equal-weight universe and form six balanced groups. These are constructed research baskets, not historical sectors or traded ETFs. Each begins at 100 with equal initial value in its members and keeps fixed shares. Missing quotes retain the last observed price, with quote ages and stale fractions reported. For basket index V_b, the target is

$$r_{b,t} = V_b(t+5)/V_b(t) - 1,$$

where the offset counts available market sessions. No future-survival screen or future price fill determines membership or the target.

2. Investigations produce candidates, not stories. A deterministic store exposes returns, volatility, volume, quote age, basket breadth and dispersion, and market-index returns. The common provisional forecast is a ridge regression over all features, fitted on at most 252 earlier origins whose labels have matured. Training-only scaling, clipping and a fixed penalty are shared by the baseline and candidate fits. Arithmetic stays in numerical code.

Six first-pass readers inspect different slices of the same universe. A planner combines their observations and freezes an investigation plan (CP1). A query chooses a target basket, any source basket or the whole universe or index group, a feature family and a lag of 0, 1, 5 or 20 sessions. Up to six queries are allowed. The tool fits the selected feature subset and reports a candidate forecast plus its gain over the common baseline on eight earlier rolling validation origins. Every validation fit excludes labels maturing after that validation cutoff.

Revision within a forecast. A final model selects one eligible candidate or the baseline for each basket. The complete vector is frozen at CP2. CP1 remains an immutable record, but its hypotheses need not survive investigation. Uninvestigated targets retain their baseline. The tool validates candidate identity and target coverage; the language model cannot supply an unsupported numerical forecast. A validation gain may reflect selection noise, so retaining the baseline is a legitimate decision.

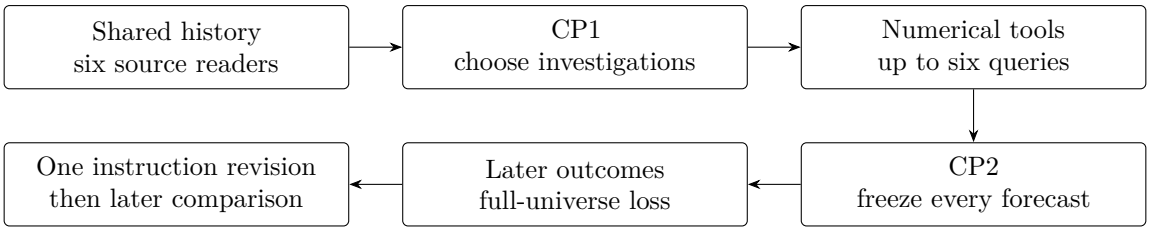

Figure 1. The executable basket loop. Forecast-time numerical tools see only cutoff-valid data. Development outcomes can guide a revision; later test outcomes cannot enter its instruction.

3. Comparison and learning. Four development dates in 2023 compare directed investigation with an undirected control. The latter runs the same reader/planner architecture but substitutes six predeclared own-basket trend queries. Both have the same maximum query allowance. Their first-pass model calls are independent, so the pilot does not isolate all sampling variability.

Evolve reads development losses and saved traces, then proposes one addition of at most 512 characters to the planner instruction. The revision is hashed and frozen before held-out execution. This learns a procedural instruction; it does not change model weights, fit a neural controller or identify market causality. One proposal is tested, including the possibility of no change.

Six later dates in 2024 compare the incumbent, its revision, the undirected arm and a serial controller. The serial controller receives all first-pass numerical rows and the same query catalogue. It chooses investigations and then selects forecasts in two calls, versus eight for each graph arm. The same inexpensive base model is used throughout. Numerical ridge and zero-return baselines require no language-model calls. A serial programme could execute the same computation graph; exclusive expressive capability is not the claim.

Every arm commits all six forecasts before the evaluator opens that date's outcome. Squared fractional return error is averaged equally over targets and origins. A failed run retains the entire numerical baseline vector, stays in the score and is separately reported. The revision is retained only if it lowers mean held-out loss, does not increase worst-origin loss and no model arm fails. A shared cap of $2 and 260 attempted calls bounds the whole experiment.

Measured result. The comparison completed 221 calls to Ling 3.0 Flash for a reported $0.021791. Directed, revised and serial held-out forecasts equalled the ridge baseline: mean squared return error 0.00065565, versus 0.00065196 for undirected investigations and 0.00064589 for zero return. Eight of 32 model arms failed candidate-to-target validation and retained the predeclared baseline fallback. The proposed instruction was not retained. There is no observed advantage for the directed graph or revision in this pilot. The complete forecast and failure records are published in `examples/baskets/`; the proposed diagnosis remains unverified model output.

4. What this experiment can establish. A completed run demonstrates a reproducible sequence from historical data through provisional reasoning, targeted computation, forecast revision, later scoring and one tested instruction change. A lower observed loss would be a descriptive result on these dates. Six held-out dates are not 36 independent experiments: baskets share constituents' market environment and errors within a date are dependent. The present sample does not support a claim of statistical superiority.

The open financial system need not be fully observed for forecasting to be scored. It does, however, limit explanations. Prices and volume alone omit many possible causes. The published data have unknown historical release timestamps and adjustment vintages; stale marks are not executable prices; instrument types and historical classifications are incomplete. Consequently the target is a retrospective last-quote research index, not trading returns or a certified point-in-time backtest. Pretrained models may also have encountered the historical period. A prospective test is needed to address that limitation.

The architecture must survive stronger comparisons. An adaptive serial model can read the same data and ask the same questions; a numerical model can summarise many histories cheaply. Zenith is useful only if its investigation policy adds value at an acceptable cost. Larger chronological tests should include repeated unchanged controls, broader numerical baselines, cost-matched alternatives and predeclared uncertainty estimates at the forecast-date level. Any procedure selected after inspecting the current test results belongs to development in its next evaluation.

Related work and scope. DSPy composes language-model modules and optimises programmes against a metric [2]. MIPRO optimises instructions and demonstrations across multi-stage programmes [3]. Zenith applies the same general principle of measured programme revision to a bounded forecasting investigation policy. The specific contribution of this prototype is the executable comparison: all-source access, numerical candidate generation, immutable provisional and final records, full-target scoring, an inexpensive serial competitor, and a separate test of one proposed instruction. Prompt revision alone is not novel, and the structure of the workflow does not establish predictive benefit.

Reproduction and evidence. The backend uses the existing native graph engine and its budgeted OpenRouter adapter. Source files, import outputs, numerical snapshots, forecast commitments and policy revisions carry hashes. Failed imports and paid attempts are retained; no failed forecast is silently removed or replaced by a fresh attempt. The numerical store is separate from the bounded text shown to readers. The query catalogue, ridge specification, exact dates and data qualifications are documented in `docs/baskets.md`; `docs/status.md` owns current results.

The previous invented System A example remains a synthetic protocol fixture. Its preparation receipt is not evidence for this financial experiment. Source and reproduction commands are in the Zenith repository.

References

[1] SocrateAI. *NEPSE Open Data*, pinned source `497e27238a10`, 2026. Dataset-specific MIT-form licence; copyright 2025 SocrateAI.

[2] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR*, 2024. arXiv:2310.03714.

[3] K. Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *EMNLP*, 2024. arXiv:2406.11695.